

BoxSim Algorithm

Binary Vector Similarity Search via Locality-Sensitive Hashing

Abstract. BoxSim transforms high-dimensional numeric vectors into compact binary codes and groups them into *boxes* based on shared bit-prefix patterns. Vectors sharing the same box are returned as approximate nearest-neighbour candidates. The design is inspired by Locality-Sensitive Hashing (LSH) and prioritises interpretability and low computational overhead over exhaustive accuracy. Preprocessing costs $\mathcal{O}(ND + N \log N)$; query lookup is $\mathcal{O}(1)$ (hash) plus a linear scan over the matched box.

Algorithm

Step-by-Step Description

Step 1. Load Data. Read an $N \times D$ matrix $\mathbf{X} \in \mathbb{R}^{N \times D}$ where each row is a D -dimensional vector and each column represents one feature dimension.

Step 2. L2 Normalisation. Convert every row to a unit-length vector so that subsequent comparisons measure *direction* rather than magnitude:

$$\hat{\mathbf{x}}_i = \frac{\mathbf{x}_i}{\|\mathbf{x}_i\|_2}, \quad \text{skip if } \|\mathbf{x}_i\|_2 = 0.$$

Step 3. Binarisation. Apply the sign function element-wise to produce a binary matrix $\mathbf{B} \in \{0, 1\}^{N \times D}$:

$$B_{ij} = \begin{cases} 1 & \hat{x}_{ij} > 0 \\ 0 & \hat{x}_{ij} \leq 0 \end{cases}$$

Hamming distance on $\mathbf{B}$ approximates cosine distance on the original vectors.

Step 4. Column Variance Scoring. Compute the population variance of each bit column j :

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (B_{ij} - \bar{B}_j)^2, \quad \bar{B}_j = \frac{1}{N} \sum_{i=1}^N B_{ij}.$$

A *low*-variance column is nearly constant and therefore more discriminative as a prefix bit.

Step 5. Reorder Columns. Sort columns in ascending order of σ_j^2 to produce $\mathbf{B}' = \mathbf{B}[\cdot, \pi]$, where π is the sorting permutation. The most stable bits now occupy the leading positions.

Step 6. Sort Rows Lexicographically. Sort the rows of $\mathbf{B}'$ in lexicographic (binary) order to obtain $\hat{\mathbf{B}}$. Vectors sharing the same binary prefix are now contiguous.

Step 7. Form Boxes (prefix length k). Group rows of $\hat{\mathbf{B}}$ that share the same first k bits into a *box*. Store the mapping $\text{prefix} \mapsto \{\text{vector indices}\}$ in a hash map. Smaller k yields fewer, larger boxes; larger k yields more, purer boxes.

Step 8. Query. Given query $\mathbf{q}$: normalise, binarise, apply permutation π , extract its k -bit prefix, and return the corresponding box as the candidate set.

Properties

Property	Detail
Distance metric	Hamming distance on $\mathbf{B}$ approximates cosine similarity after L2 normalisation.
Precision vs. recall	Larger $k \Rightarrow$ smaller, purer boxes (high precision, lower recall) and vice versa.
Scalability	Box lookup is $\mathcal{O}(1)$; only the matched box is scanned.
Limitation	Binary quantisation discards magnitude; works best when direction encodes similarity.

BoxSim — Pseudocode

Formal notation for implementation reference

Pseudocode

Algorithm 1 BoxSim Preprocessing

Input: Matrix $\mathbf{X} \in \mathbb{R}^{N \times D}$
Output: Sorted binary matrix $\hat{\mathbf{B}}$, column permutation π

```

1: Function BOXSIM_PREPROCESS( $\mathbf{X}$ )
2:   // — Step 1: L2 Normalisation —
3:   for  $i \leftarrow 0$  to  $N - 1$  do
4:      $n_i \leftarrow \|\mathbf{x}_i\|_2$ 
5:      $\hat{\mathbf{x}}_i \leftarrow \mathbf{x}_i / n_i$  if  $n_i > 0$  else  $\mathbf{x}_i$ 
6:   end for
7:   // — Step 2: Binarisation —
8:   for  $i \leftarrow 0$  to  $N - 1$  do
9:     for  $j \leftarrow 0$  to  $D - 1$  do
10:       $B_{ij} \leftarrow (\hat{x}_{ij} > 0) ? 1 : 0$ 
11:    end for
12:  end for
13:  // — Step 3: Column Variance Scoring —
14:  for  $j \leftarrow 0$  to  $D - 1$  do
15:     $\bar{B}_j \leftarrow \frac{1}{N} \sum_i B_{ij}$ 
16:     $\sigma_j^2 \leftarrow \frac{1}{N} \sum_i (B_{ij} - \bar{B}_j)^2$ 
17:  end for
18:  // — Step 4: Reorder Columns by Ascending Variance —
19:   $\pi \leftarrow \text{ARGSORT}(\sigma^2, \text{ascending})$ 
20:   $\mathbf{B}' \leftarrow \mathbf{B}[:, \pi]$ 
21:  // — Step 5: Sort Rows Lexicographically —
22:   $\hat{\mathbf{B}} \leftarrow \text{LEXSORT}(\mathbf{B}', \text{axis} = \text{rows})$ 
23:  return  $\hat{\mathbf{B}}, \pi$ 
23: end Function
```

Algorithm 2 Box Formation

Input: Sorted binary matrix $\hat{\mathbf{B}}$, prefix length $k \in \{1, \dots, D\}$
Output: Hash map $\mathcal{M} : \text{prefix} \mapsto \text{list of indices}$

```

1: Function FORM_BOXES( $\hat{\mathbf{B}}, k$ )
2:    $\mathcal{M} \leftarrow \{\}$ 
3:   for  $i \leftarrow 0$  to  $N - 1$  do
4:      $p \leftarrow \text{JOIN}(\hat{B}_{i,0}, \hat{B}_{i,1}, \dots, \hat{B}_{i,k-1})$  ▷ e.g.  $[1, 0, 1] \rightarrow$  “101”
5:      $\mathcal{M}[p].\text{APPEND}(i)$ 
6:   end for return  $\mathcal{M}$ 
7: end Function
```

Algorithm 3 BoxSim Query**Input:** Query $\mathbf{q} \in \mathbb{R}^D$, permutation π , map $\mathcal{M}$, prefix length k **Output:** Candidate index set $\mathcal{C}$

```

1: Function BOXSIM_QUERY( $\mathbf{q}$ ,  $\pi$ ,  $\mathcal{M}$ ,  $k$ )
2:    $n \leftarrow \|\mathbf{q}\|_2$ 
3:    $\hat{\mathbf{q}} \leftarrow \mathbf{q}/n$  if  $n > 0$  else  $\mathbf{q}$ 
4:   for  $j \leftarrow 0$  to  $D - 1$  do
5:      $b_j \leftarrow (\hat{q}_j > 0) ? 1 : 0$ 
6:   end for
7:    $\mathbf{b}' \leftarrow \mathbf{b}[\pi]$  ▷ apply same column permutation
8:    $p \leftarrow \text{JOIN}(b'_0, \dots, b'_{k-1})$ 
9:    $\mathcal{C} \leftarrow \mathcal{M}.\text{GET}(p, \emptyset)$  ▷  $\mathcal{O}(1)$  hash lookup return  $\mathcal{C}$ 
10: end Function

```

Notation Reference

Symbol	Meaning
N, D	Number of vectors (rows) and dimensions (columns).
$\mathbf{X}$	Original input matrix $\in \mathbb{R}^{N \times D}$.
$\hat{\mathbf{X}}$	L2-normalised version of $\mathbf{X}$ (unit-length rows).
$\mathbf{B}$	Binary matrix $\in \{0, 1\}^{N \times D}$ derived from $\hat{\mathbf{X}}$.
σ_j^2	Population variance of column j of $\mathbf{B}$.
π	Permutation sorting columns by ascending σ_j^2 .
$\mathbf{B}'$	$\mathbf{B}$ with columns reordered by π .
$\hat{\mathbf{B}}$	$\mathbf{B}'$ with rows sorted lexicographically.
k	Prefix length: number of leading bits defining a box.
p	k -bit string key (prefix), e.g. “101”.
$\mathcal{M}$	Hash map: prefix $\mapsto$ list of row indices.
$\mathcal{C}$	Candidate set (approximate neighbours) returned for a query.

Complexity summary. Preprocessing: $\mathcal{O}(ND)$ for normalisation and binarisation, $\mathcal{O}(D \log D)$ for column sort, $\mathcal{O}(N \log N)$ for row sort. Query: $\mathcal{O}(D)$ to encode + $\mathcal{O}(1)$ hash lookup + $\mathcal{O}(|\mathcal{C}|)$ scan of the matched box.